Homework 2: Website Development & AI Agent Planning

Aegis — AI-Driven Phishing & Threat Analysis Center

Advanced Web Development
Batur Dinçer

March 8, 2026

Abstract

This document presents Aegis, a React-based web application designed to simulate a professional Security Operations Center (SOC) tool for threat analysis. Users can submit suspicious files or URLs and receive comprehensive, AI-generated threat reports including static analysis, sandbox behavioral simulation, and threat intelligence cross-referencing. This document outlines the project overview, technical implementation, AI agent concept for future integration, and system architecture design.

Contents

1 Project Overview **3**
1.1 Website Topic and Purpose . . . 3
1.2 Target Users . . . 3
1.3 Core Features . . . 3
1.3.1 File Analysis . . . 3
1.3.2 URL Analysis . . . 4
1.3.3 Three-Phase Analysis Pipeline . . . 4
1.3.4 Risk Scoring System . . . 4
1.3.5 Detailed Threat Reports . . . 4
1.3.6 Scan History . . . 4
1.3.7 Professional UI/UX . . . 5

2 Technical Implementation **6**
2.1 Technology Stack . . . 6
2.2 Project Structure . . . 6
2.3 Key Implementation Details . . . 6
2.3.1 Mock Analysis Engine . . . 6
2.3.2 State Management . . . 7
2.3.3 Data Persistence . . . 7

3 AI Agent Concept (Future Integration) **8**
3.1 The Security Analyst Agent . . . 8
3.2 Problem Statement . . . 8
3.3 Agent Type . . . 8
3.4 User Interaction Model . . . 8
3.5 Agent Workflow: The Analytical Pipeline . . . 9
3.5.1 Phase 1: Static Analysis (Pre-Execution) . . . 9
3.5.2 Phase 2: Dynamic Analysis (Sandbox Simulation) . . . 9

3.5.3 Phase 3: Threat Intelligence . . . 9
3.5.4 Phase 4: Synthesis and Reporting . . . 9
3.6 Future AI Integration: Groq LLM . . . 10

4 System Architecture **11**
4.1 Current Architecture (Draft Version) . . . 11
4.2 Planned Architecture (AI Agent Integration) . . . 11
4.2.1 Frontend Layer . . . 12
4.2.2 Backend Layer (API Gateway) . . . 12
4.2.3 AI Agent Layer . . . 12
4.2.4 LLM Synthesizer . . . 13
4.2.5 Data Layer . . . 13
4.2.6 External Services . . . 13
4.3 Data Flow . . . 13
4.4 Security Considerations . . . 14
4.5 Deployment Strategy . . . 14

5 Installation and Usage **15**
5.1 Prerequisites . . . 15
5.2 Installation Steps . . . 15
5.3 Build for Production . . . 15
5.4 Available Scripts . . . 15

6 Future Roadmap **15**
6.1 Phase 1: Backend Integration (Next Assignment) . . . 15
6.2 Phase 2: AI Agent Implementation . . . 15
6.3 Phase 3: Advanced Features . . . 16

7 Conclusion **16**
7.1 Key Achievements . . . 16
7.2 Learning Outcomes . . . 16

1 Project Overview

1.1 Website Topic and Purpose

Aegis is a web-based cybersecurity platform designed to provide individuals and organizations with a fast, accessible interface for detecting malicious files and URLs. The platform simulates the workflow of a professional Security Operations Center (SOC) tool, combining multiple analysis layers into a single, coherent user experience.

The project addresses a critical real-world need: threats such as phishing links, ransomware droppers, and trojanized documents are distributed to end-users daily. Most available scanning tools are either:

- Command-line only, requiring technical expertise
- Locked behind enterprise paywalls
- Provide minimal explanation of findings
- Lack actionable mitigation guidance

Aegis aims to be **approachable**, **explainable**, and **actionable** for users of all technical backgrounds.

1.2 Target Users

The platform is designed to serve multiple user segments with distinct needs:

User Segment	Use Case
Cybersecurity Researchers	Rapid IOC (Indicator of Compromise) enrichment and behavioral baseline comparison
IT/Security Departments	First-response triage of suspicious email attachments or links
Security-Aware Individuals	Self-service scanning before opening unknown files
Students & Educators	Hands-on demonstration of threat analysis concepts and security workflows

Table 1: Target user segments and their use cases

1.3 Core Features

The current implementation (draft version) includes the following functional features:

1.3.1 File Analysis

Users can drag-and-drop binary and script files for analysis. Supported file types include:

- Executables: `.exe`, `.msi`, `.dmg`, `.bin`
- Scripts: `.ps1`, `.bat`, `.vbs`, `.js`, `.py`, `.sh`, `.php`
- Documents: `.pdf`, `.docx`
- Archives: `.zip`, `.iso`
- Libraries: `.dll`, `.so`, `.jar`

Files undergo multi-phase inspection with results presented in a structured format.

1.3.2 URL Analysis

Real-time scanning of suspicious web addresses for:

- Phishing patterns (typosquatting, homograph attacks)
- Redirect chain analysis
- Injection attempt detection (XSS, SQLi patterns)
- Domain reputation checking

1.3.3 Three-Phase Analysis Pipeline

Each submission undergoes three distinct analytical phases:

1. **Static Analysis** — Pre-execution inspection of file structure, entropy, embedded strings, and API call patterns
2. **Dynamic Analysis** — Simulated runtime behavior analysis in an isolated sandbox environment
3. **Threat Intelligence** — Cross-referencing against global reputation databases and known attack pattern libraries

1.3.4 Risk Scoring System

A weighted composite score (0–100) synthesizes results from all three phases:

$$RiskScore = 0.35 \times S + 0.40 \times D + 0.25 \times T$$

where:

- S = Static analysis score
- D = Dynamic analysis score
- T = Threat intelligence score

The final score produces one of three verdicts:

- **CLEAN** (0–39): No significant threats detected
- **SUSPICIOUS** (40–69): Potentially unwanted behavior identified
- **MALICIOUS** (70–100): Clear malicious intent detected

1.3.5 Detailed Threat Reports

Each analysis generates a comprehensive report featuring:

- Circular SVG risk gauge visualization
- Phase-by-phase collapsible findings with technical details
- Natural-language descriptions for non-expert users
- Prioritized mitigation recommendations
- Timestamp and file metadata

1.3.6 Scan History

All analyses are persisted to browser LocalStorage, enabling:

- Historical tracking of all scans
- Search and filter capabilities
- Drill-down into past reports
- Trend analysis across multiple submissions

1.3.7 Professional UI/UX

The interface employs a dark-mode SOC aesthetic designed to feel professional and trustworthy:

- Responsive layout for desktop and mobile
- Intuitive navigation with React Router
- Loading states and progress indicators
- Accessible color contrast and typography

2 Technical Implementation

2.1 Technology Stack

The project is built using modern web technologies optimized for performance and developer experience:

Category	Technology
Frontend Framework	React 18
Build Tool	Vite 7
Styling	Tailwind CSS v3 (dark theme)
Routing	React Router v6
Icons	Lucide React
Analysis Engine	Custom deterministic mock (mockAnalysis.js)
Data Persistence	Browser LocalStorage
Version Control	Git

Table 2: Technology stack components

2.2 Project Structure

```
advancedwebhw1/
          index.html              # Entry point
          package.json            # Dependencies
          vite.config.js          # Build configuration
          tailwind.config.js      # Styling configuration
          eslint.config.js        # Code quality rules
          postcss.config.js       # CSS processing
          public/                 # Static assets
          src/
              main.jsx           # React entry point
              App.jsx            # Root component
              index.css          # Global styles
              components/
                    Navbar.jsx     # Navigation component
                    Footer.jsx     # Footer component
                    RiskGauge.jsx  # SVG risk visualization
                    ThreatReport.jsx # Report modal
              pages/
                    HomePage.jsx   # Landing page
                    DashboardPage.jsx # Analysis interface
                    HistoryPage.jsx   # Scan history
              utils/
                  mockAnalysis.js # Mock analysis engine
                  storage.js      # LocalStorage utilities
```

Listing 1: File system structure

2.3 Key Implementation Details

2.3.1 Mock Analysis Engine

The current version uses a deterministic client-side analysis engine (`mockAnalysis.js`) that simulates the full agent pipeline without requiring backend services. This approach enables:

- Rapid prototyping and development
- Zero-cost deployment (static hosting)

- Offline functionality
- Predictable behavior for testing

The mock engine analyzes file extensions, names, sizes, and URL patterns to generate realistic threat reports with weighted scoring.

2.3.2 State Management

The application uses React hooks (`useState`, `useEffect`, `useCallback`) for local state management, avoiding the complexity of external state libraries for this scale.

2.3.3 Data Persistence

Scan results are serialized to JSON and stored in browser LocalStorage with a key-based structure:

```
localStorage.setItem('aegis-reports',
    JSON.stringify(reports))
```

3 AI Agent Concept (Future Integration)

3.1 The Security Analyst Agent

The "Security Analyst Agent" will serve as the cognitive engine of Aegis. Unlike traditional signature-based scanners that merely ask *"Is this known?"*, the agent will reason about **intent** and **behavior** by asking *"What is this trying to do?"*

This distinction is critical: novel malware variants, zero-day exploits, and polymorphic threats evade signature databases by design. A reasoning agent that analyzes structure, behavior, and context is significantly harder to evade.

3.2 Problem Statement

> *How can a non-expert user obtain a reliable, explainable threat assessment for an unknown file or URL in under 10 seconds — without installing specialist tools?*

The agent solves this by encapsulating the multi-step analytical workflow of a human security analyst into an automated pipeline.

3.3 Agent Type

The agent is designed as a **multi-step analytical pipeline agent** with three specialized sub-components:

1. **Inspector** (Static Analysis Sub-Agent)
   Reasons about file structure and content without execution
2. **Observer** (Dynamic Analysis Sub-Agent)
   Reasons about runtime behavior in an isolated sandbox environment
3. **Evaluator** (Threat Intelligence Sub-Agent)
   Reasons about external reputation data and known attack patterns

A **Synthesizer Module** reconciles the three sub-agent outputs into a final risk score and comprehensive report.

3.4 User Interaction Model

Users interact with the agent entirely through the web UI, without needing to understand the underlying agent architecture. The interaction flow is:

1. User submits file or URL via drag-and-drop or form input
2. Input validation ensures file type and size constraints
3. Agent pipeline is triggered automatically
4. Real-time progress updates show which phase is executing
5. Results are presented in a modal with:
   - Circular risk gauge (0–100)
   - Phase-by-phase collapsible findings
   - Natural-language mitigation recommendations

3.5 Agent Workflow: The Analytical Pipeline

3.5.1 Phase 1: Static Analysis (Pre-Execution)

The Inspector sub-agent performs inspection *without* executing the target:

- **Hash Verification**: SHA-256 hash generated and cross-referenced against known malicious sample databases (e.g., VirusTotal, MalwareBazaar)
- **Entropy Analysis**: High entropy in binary sections indicates obfuscation or packing — a common evasion technique
- **API Call Pattern Detection**: Identification of suspicious import patterns (e.g., `CreateRemoteThread`, `WriteProcessMemory`, `VirtualAllocEx`) that suggest code injection capability
- **String Extraction**: Searching for embedded IPs, URLs, registry paths, and encoded payloads
- **PE/ELF Header Analysis**: Inspection of executable format structures for anomalies

Output: Static risk score (0–100) with list of suspicious indicators

3.5.2 Phase 2: Dynamic Analysis (Sandbox Simulation)

The Observer sub-agent reasons about runtime behavior:

- **Behavioral Prediction**: AI model predicts likely runtime actions based on file characteristics
- **Resource Access Patterns**: Analysis of potential file system, registry, and network operations
- **Process Injection Detection**: Identification of parent-child process relationships
- **Anti-Analysis Technique Detection**: Recognition of VM/debugger detection attempts

In future implementations, this phase will integrate with containerized sandbox environments (Docker) for actual execution in isolation.

Output: Dynamic risk score (0–100) with behavioral findings

3.5.3 Phase 3: Threat Intelligence

The Evaluator sub-agent queries external reputation databases:

- **VirusTotal API**: Multi-engine antivirus scanning results
- **AbuseIPDB**: IP reputation and abuse reports
- **URLhaus/PhishTank**: Known malicious URL databases
- **MITRE ATT&CK Mapping**: Classification into known attack tactics/techniques

Output: Intelligence risk score (0–100) with reputation data

3.5.4 Phase 4: Synthesis and Reporting

The Synthesizer module:

- Calculates weighted composite risk score: $0.35S + 0.40D + 0.25T$
- Assigns verdict (CLEAN / SUSPICIOUS / MALICIOUS)
- Generates prioritized mitigation steps based on findings
- Formats results into human-readable report

3.6 Future AI Integration: Groq LLM

The planned AI agent will leverage **Groq Cloud API** with the **Llama 3.3 70B** model for:

- Natural language interpretation of analysis results
- Context-aware mitigation recommendation generation
- User query handling ("What does this finding mean?")
- Report summarization for non-technical users

The LLM will act as the "reasoning layer" that translates technical indicators into actionable insights.

4 System Architecture

4.1 Current Architecture (Draft Version)

The current implementation is a **client-side static web application**:

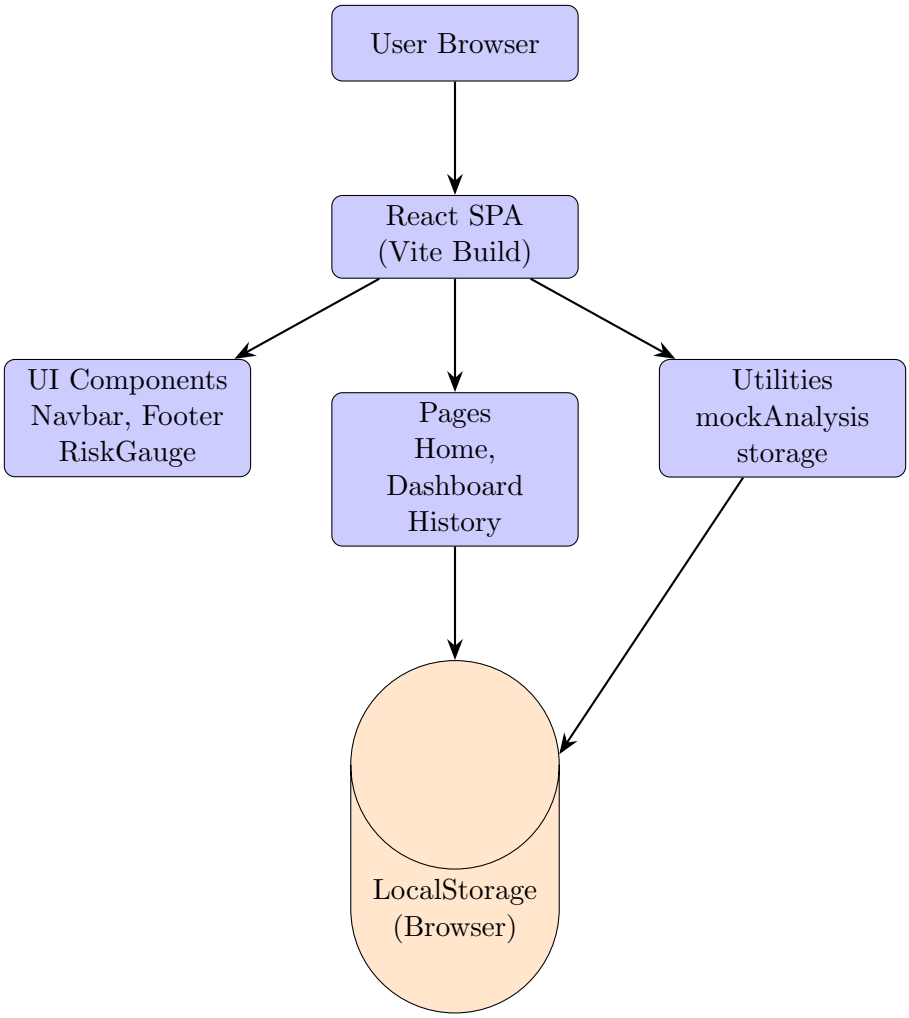

Figure 1: Current client-only architecture

Architecture Components:

- **User Browser**: Executes the entire application
- **React SPA**: Frontend framework managing routing and state
- **Mock Analysis Engine**: Deterministic JavaScript-based threat simulation
- **LocalStorage**: Persistent client-side data storage

Limitations:

- No real-time threat intelligence lookups
- No server-side processing or database
- Limited to deterministic analysis patterns
- Cannot perform actual sandboxed execution

4.2 Planned Architecture (AI Agent Integration)

The future architecture will implement a **client-server model with AI agent backend**:

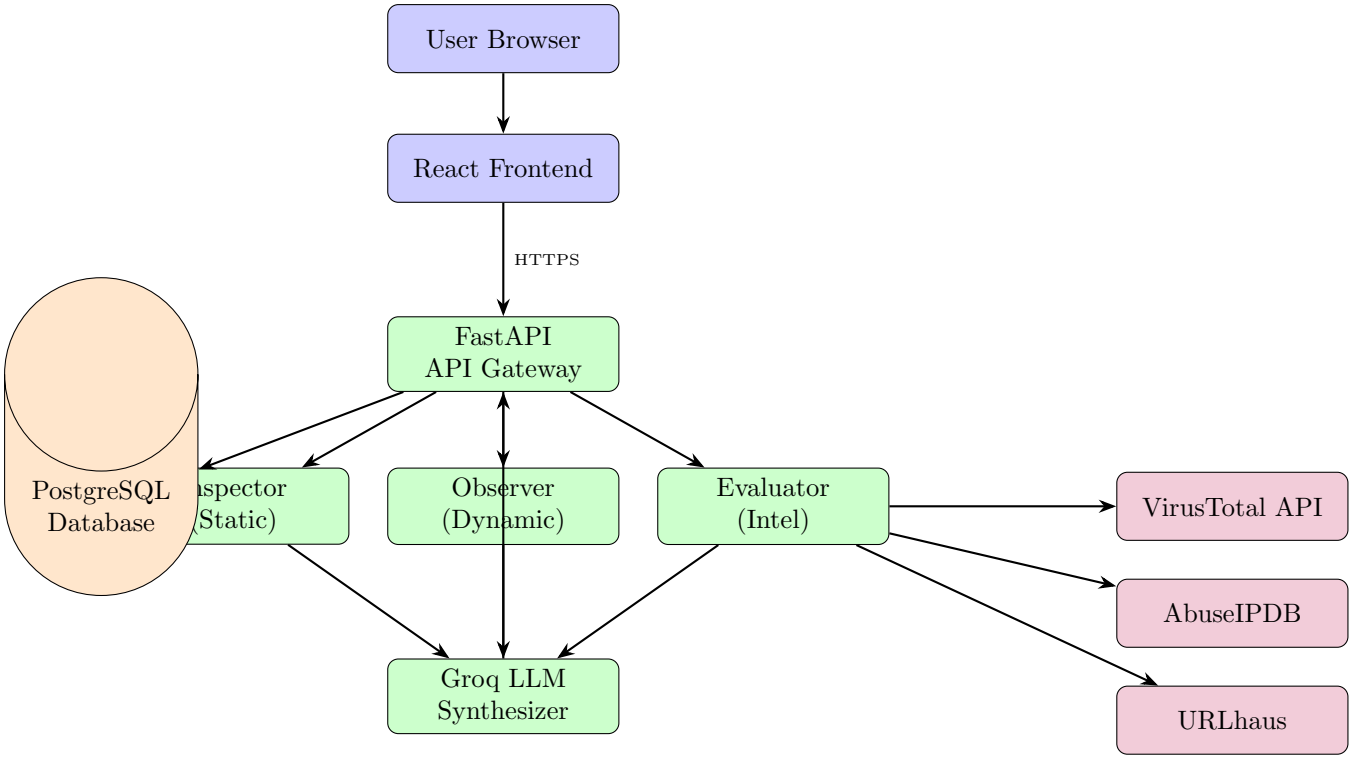

Figure 2: Planned AI agent architecture

4.2.1 Frontend Layer

- React SPA (unchanged UI)
- Communicates with backend via RESTful API
- WebSocket connection for real-time progress updates

4.2.2 Backend Layer (API Gateway)

- **Framework**: FastAPI (Python)
- **Responsibilities**:
  - File upload handling
  - Analysis pipeline orchestration
  - Authentication & rate limiting
  - Result caching

4.2.3 AI Agent Layer

Three specialized sub-agents coordinated by the API gateway:

1. **Inspector (Static Analysis)**
   - Python libraries: `pefile`, `capstone`, `yara`
   - Hash calculation, entropy analysis
   - String extraction and regex matching
2. **Observer (Dynamic Analysis)**
   - Containerized sandbox: Docker + Cuckoo Sandbox
   - Behavioral monitoring in isolated environment

    - Process, network, and file system telemetry

3. **Evaluator (Threat Intelligence)**
    - External API integrations: VirusTotal, AbuseIPDB, URLhaus
    - MITRE ATT&CK technique mapping
    - Reputation database queries

4.2.4 LLM Synthesizer

- **Model**: Groq Llama 3.3 70B via Cloud API
- **Functions**:
  - Correlate findings from all three sub-agents
  - Generate natural-language report summaries
  - Produce prioritized mitigation recommendations
  - Answer user follow-up questions

4.2.5 Data Layer

- **Database**: PostgreSQL
- **Stored Data**:
  - User accounts and authentication tokens
  - Scan history with full reports
  - Cached threat intelligence results
  - Performance metrics and analytics

4.2.6 External Services

Third-party APIs for threat intelligence enrichment:

- **VirusTotal**: Multi-engine antivirus scanning
- **AbuseIPDB**: IP reputation checking
- **URLhaus**: Malicious URL database
- **PhishTank**: Phishing URL verification

4.3 Data Flow

Analysis Request Flow:

1. User uploads file/URL via React frontend
2. Frontend sends HTTPS POST request to FastAPI
3. API Gateway validates input and creates analysis job
4. Three sub-agents execute in parallel:
    - Inspector analyzes file structure
    - Observer prepares sandbox environment
    - Evaluator queries external threat intel APIs
5. Sub-agent results sent to Groq LLM Synthesizer
6. LLM generates unified report with:

    - Composite risk score
    - Natural-language summary
    - Mitigation recommendations

7. Results stored in PostgreSQL
8. Final report returned to frontend via API
9. Frontend displays report in modal with visualizations

4.4 Security Considerations

- **File Uploads**: Sandboxed execution in isolated Docker containers
- **API Authentication**: JWT tokens with expiration
- **Rate Limiting**: Prevent abuse of analysis pipeline
- **Data Encryption**: TLS 1.3 for all API communications
- **Input Validation**: File type and size restrictions
- **Secrets Management**: Environment variables for API keys

4.5 Deployment Strategy

Current (Draft):

- Static hosting on GitHub Pages, Vercel, or Netlify
- Zero-cost infrastructure
- Global CDN distribution

Future (AI Agent):

- Frontend: Vercel/Netlify (static hosting)
- Backend: AWS EC2 or DigitalOcean Droplet
- Database: Managed PostgreSQL (AWS RDS or Supabase)
- Sandbox: Dedicated isolation server with Docker
- LLM: Groq Cloud API (pay-per-token)

5 Installation and Usage

5.1 Prerequisites

- Node.js 18+ and npm 9+
- Modern web browser (Chrome, Firefox, Edge, Safari)

5.2 Installation Steps

```
# Clone the repository
git clone https://github.com/yourusername/aegis-threat-analysis.git
cd aegis-threat-analysis

# Install dependencies
npm install

# Start development server
npm run dev
```

Listing 2: Installation commands

The application will be available at `http://localhost:5173`

5.3 Build for Production

```
# Create optimized production build
npm run build

# Preview production build locally
npm run preview
```

Listing 3: Production build

The build output will be in the `dist/` directory, ready for deployment.

5.4 Available Scripts

- `npm run dev` — Start development server with hot reload
- `npm run build` — Create production-optimized build
- `npm run preview` — Preview production build locally
- `npm run lint` — Check code quality with ESLint

6 Future Roadmap

6.1 Phase 1: Backend Integration (Next Assignment)

- Implement FastAPI backend server
- Set up PostgreSQL database
- Migrate from mock analysis to server-side processing
- Add user authentication

6.2 Phase 2: AI Agent Implementation

- Integrate Groq LLM for report generation
- Implement static analysis sub-agent with `pefile` and `yara`
- Set up Docker sandbox environment for dynamic analysis
- Connect threat intelligence APIs (VirusTotal, AbuseIPDB)

6.3 Phase 3: Advanced Features

- Real-time collaborative analysis (multiple users)
- Custom YARA rule creation
- Batch file analysis
- API endpoint for programmatic access
- Integration with SIEM platforms
- Mobile application (React Native)

7 Conclusion

Aegis represents a modern approach to democratizing cybersecurity threat analysis. By combining intuitive web interfaces with sophisticated AI-driven analysis pipelines, the platform makes advanced security capabilities accessible to users of all technical backgrounds.

The current draft implementation successfully demonstrates the core UI/UX concepts and user workflows. The modular architecture is designed for seamless integration with AI agents and external services in future iterations.

This project serves as both a practical security tool and a demonstration of how AI agents can augment human decision-making in complex, high-stakes domains like cybersecurity.

7.1 Key Achievements

- Functional multi-page React application
- Three-phase analysis workflow simulation
- Professional SOC-inspired UI design
- Persistent scan history with LocalStorage
- Comprehensive documentation and planning
- Production-ready build system

7.2 Learning Outcomes

- Modern React development patterns and hooks
- Component-based architecture design
- Client-side state management strategies
- Responsive UI/UX implementation with Tailwind CSS
- Planning AI agent integration into web applications
- System architecture design for scalable applications